



Control Statements: Part 2; Logical Operators

C++ How to Program, 9/e



5.1 Introduction

- ▶ `for`, `do...while` and `switch` statements.
- ▶ counter-controlled repetition.
- ▶ Introduce the `break` and `continue` program control statements.
- ▶ Logical operators for more powerful conditional expressions.
- ▶ Examine the common error of confusing the equality (`==`) and assignment (`=`) operators, and how to avoid it.
- ▶ Summarize C++'s control statements.



5.2 Essentials of Counter-Controlled Repetition

- ▶ Counter-controlled repetition requires
 - the **name of a control variable** (or loop counter)
 - the **initial value** of the control variable
 - the **loop-continuation condition** that tests for the **final value** of the control variable (i.e., whether looping should continue)
 - the **increment** (or **decrement**) by which the control variable is modified each time through the loop.



5.2 Essentials of Counter-Controlled Repetition

- ▶ The program in Fig. 5.1 prints the numbers from 1 to 10. The declaration in line 8 *names* the control variable (counter), declares it to be an unsigned `int`, reserves space for it in memory and sets it to an *initial value* of 1.
- ▶ Declarations that require initialization are *executable* statements.
- ▶ In C++, it's more precise to call a declaration that also reserves memory a **definition**.



```
1 // Fig. 5.1: fig05_01.cpp
2 // Counter-controlled repetition.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     unsigned int counter = 1; // declare and initialize control variable
9
10    while ( counter <= 10 ) // loop-continuation condition
11    {
12        cout << counter << " ";
13        ++counter; // increment control variable by 1
14    } // end while
15
16    cout << endl; // output a newline
17 } // end main
```

1 2 3 4 5 6 7 8 9 10

Fig. 5.1 | Counter-controlled repetition.



Error-Prevention Tip 5.1

Floating-point values are approximate, so controlling counting loops with floating-point variables can result in imprecise counter values and inaccurate tests for termination. Control counting loops with integer values. Separately, ++ and -- can be used only with integer operands.



5.3 for Repetition Statement

- ▶ The **for repetition statement** specifies the counter-controlled repetition details in a single line of code.
- ▶ To illustrate the power of **for**, let's rewrite the program of Fig. 5.1. The result is shown in Fig. 5.2.
- ▶ The initialization occurs once when the loop is encountered.
- ▶ The condition is tested next and each time the body completes.
- ▶ The body executes if the condition is true.
- ▶ The increment occurs after the body executes.
- ▶ Then, the condition is tested again.
- ▶ If there is more than one statement in the body of the **for**, braces are required to enclose the body of the loop.



```
1 // Fig. 5.2: fig05_02.cpp
2 // Counter-controlled repetition with the for statement.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     // for statement header includes initialization,
9     // loop-continuation condition and increment.
10    for ( unsigned int counter = 1; counter <= 10; ++counter )
11        cout << counter << " ";
12
13    cout << endl; // output a newline
14 }
```

```
1 2 3 4 5 6 7 8 9 10
```

Fig. 5.2 | Counter-controlled repetition with the for statement.



5.3 for Repetition Statement (cont.)

for Statement Header Components

- ▶ Figure 5.3 takes a closer look at the `for` statement header (line 10) of Fig. 5.2.

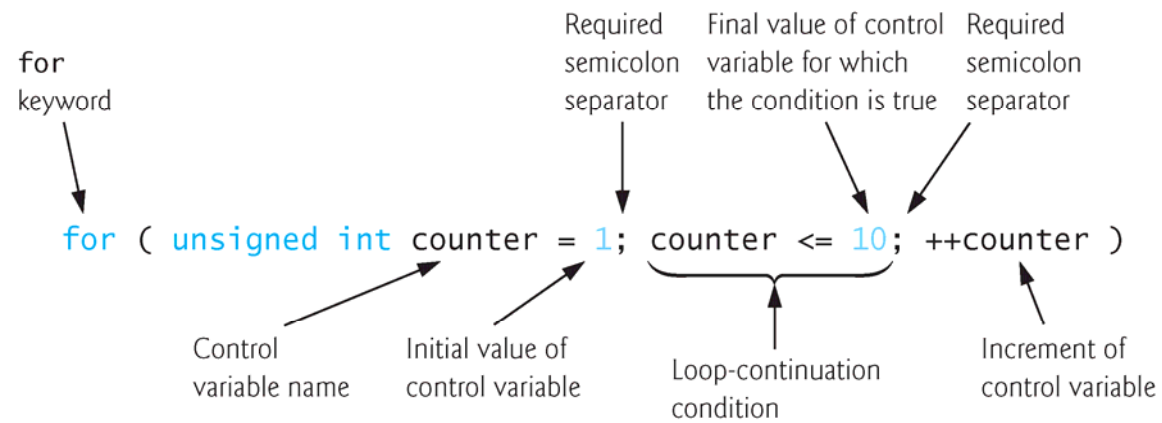


Fig. 5.3 | `for` statement header components.



Common Programming Error 5.1

Using an incorrect relational operator or using an incorrect final value of a loop counter in the condition of a `while` or `for` statement can cause off-by-one errors.



Good Programming Practice 5.1

Using the final value in the condition of a `while` or `for` statement and using the `<=` relational operator will help avoid off-by-one errors. For a loop used to print the values 1 to 10, for example, the loop-continuation condition should be `counter <= 10` rather than `counter < 10` (which is an off-by-one error) or `counter < 11` (which is nevertheless correct). Many programmers prefer so-called **zero-based counting**, in which, to count 10 times through the loop, `counter` would be initialized to zero and the loop-continuation test would be `counter < 10`.



5.3 for Repetition Statement (cont.)

General Format of a for Statement

- ▶ The general form of the **for** statement is
 - **for** (*initialization*; *loopContinuationCondition*; *increment*)
 statement
- ▶ where the *initialization* expression initializes the loop's control variable, *loopContinuationCondition* determines whether the loop should continue executing and *increment* increments the control variable.
- ▶ In most cases, the **for** statement can be represented by an equivalent **while** statement, as follows:
 - *initialization*;

```
while ( loopContinuationCondition )  
{  
    statement  
    increment;  
}
```



5.3 for Repetition Statement (cont.)

- ▶ If the *initialization* expression declares the control variable, the control variable can be used *only* in the body of the **for** statement—the control variable will be unknown *outside* the **for** statement.
- ▶ This restricted use of the control variable name is known as the variable's **scope**.
- ▶ The scope of a variable specifies *where* it can be used in a program.



5.3 for Repetition Statement (cont.)

Comma-Separated Lists of Expressions

- ▶ The *initialization* and *increment* expressions can be comma-separated lists of expressions.
- ▶ The commas, as used in these expressions, are **comma operators**, which guarantee that lists of expressions evaluate from left to right.
 - The lowest precedence of all C++ operators.

.



Good Programming Practice 5.2

Place only expressions involving the control variables in the initialization and increment sections of a **for** statement.



5.3 for Repetition Statement (cont.)

Expressions in the for Statement's Header Are Optional

- ▶ The three expressions in the **for** statement header are optional (but the two semicolon separators are *required*).
- ▶ If the *loopContinuationCondition* is omitted, C++ assumes that the condition is true, thus creating an *infinite loop*.
- ▶ One might omit the *initialization* expression if the control variable is initialized earlier in the program.
- ▶ One might omit the *increment* expression if the increment is calculated by statements in the body of the **for** or if no increment is needed.



5.3 for Repetition Statement (cont.)

Increment Expression Acts Like a Standalone Statement

- ▶ The increment expression in the **for** statement acts like a standalone statement at the end of the **for** statement's body.
- ▶ The expressions
 - `counter = counter + 1`
 - `counter += 1`
 - `++counter`
 - `counter++`
- ▶ are all equivalent in the *increment* expression (when no other code appears there).



Common Programming Error 5.2

Placing a semicolon immediately to the right of the right parenthesis of a **for** header makes the body of that **for** statement an empty statement. This is usually a logic error.



5.3 for Repetition Statement (cont.)

for Statement: Notes and Observations

- ▶ The initialization, loop-continuation condition and increment expressions of a **for** statement can contain arithmetic expressions.
- ▶ The “increment” of a **for** statement can be negative, in which case it’s really a *decrement* and the loop actually counts *downward*.
- ▶ If the loop-continuation condition is *initially false*, the body of the **for** statement is not performed.



Error-Prevention Tip 5.2

Although the value of the control variable can be changed in the body of a `for` statement, avoid doing so, because this can lead to subtle logic errors.



5.3 for Repetition Statement (cont.)

for Statement UML Activity Diagram

- ▶ The **for** repetition statement's UML activity diagram is similar to that of the **while** statement (Fig. 4.6).
- ▶ Figure 5.4 shows the activity diagram of the **for** statement in Fig. 5.2.
- ▶ The diagram makes it clear that initialization occurs once *before* the loop-continuation test is evaluated the first time, and that incrementing occurs *each time* through the loop *after* the body statement executes.

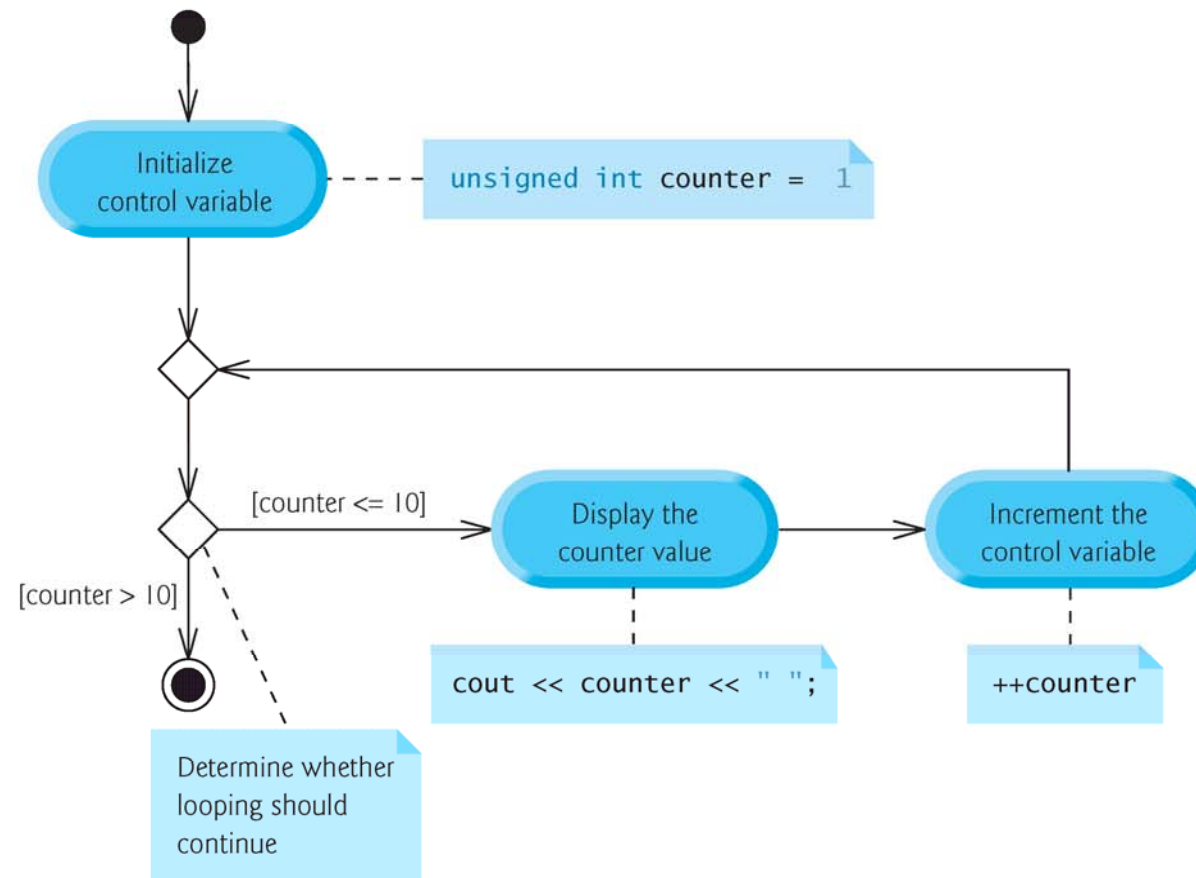


Fig. 5.4 | UML activity diagram for the for statement in Fig. 5.2.



5.4 Examples Using the for Statement

- ▶ Vary the control variable from 1 to 100 in increments of 1.
 - `for ( unsigned int i = 1; i <= 100; ++i )`
- ▶ Vary the control variable from 100 down to 0 in decrements of 1. Notice that we used type `int` for the control variable in this for header. The condition does not become false until control variable `i` contains -1, so the control variable must be able to store both positive and negative numbers.
 - `for ( int i = 100; i >= 0; --i )`
- ▶ Vary the control variable from 7 to 77 in steps of 7.
 - `for (unsigned int i = 7; i <= 77; i += 7 )`
- ▶ Vary the control variable from 20 down to 2 in steps of -2.
 - `for (unsigned int i = 20; i >= 2; i -= 2 )`

5.4 Examples Using the for Statement (cont.)



- ▶ Vary the control variable over the following sequence of values:
2, 5, 8, 11, 14, 17.
 - `for (unsigned int i = 2; i <= 17; i += 3 )`
- ▶ Vary the control variable over the following sequence of values:
99, 88, 77, 66, 55.
 - `for (unsigned int i = 99; i >= 55; i -= 11 )`



Common Programming Error 5.3

Not using the proper relational operator in the loop-continuation condition of a loop that counts downward (such as incorrectly using `i <= 1` instead of `i >= 1` in a loop counting down to 1) is a logic error that yields incorrect results when the program runs.



Common Programming Error 5.4

Do not use equality operators (`!=` or `==`) in a loop-continuation condition if the loop's control variable increments or decrements by more than 1. For example consider the for statement header `for ( unsigned int counter = 1; counter != 10; counter += 2 )`. The loop-continuation test `counter != 10` never becomes false (resulting in an infinite loop) because `counter` increments by 2 after each iteration.

5.4 Examples Using the for Statement (cont.)



Application: Summing the Even Integers from 2 to 20

- ▶ The program of Fig. 5.5 uses a **for** statement to sum the even integers from 2 to 20.



```
1 // Fig. 5.5: fig05_05.cpp
2 // Summing integers with the for statement.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     unsigned int total = 0; // initialize total
9
10    // total even integers from 2 through 20
11    for ( unsigned int number = 2; number <= 20; number += 2 )
12        total += number;
13
14    cout << "Sum is " << total << endl; // display results
15 }
```

Sum is 110

Fig. 5.5 | Summing integers with the for statement.



Good Programming Practice 5.3

Although statements preceding a `for` and statements in the body of a `for` often can be merged into the `for` header, doing so can make the program more difficult to read, maintain, modify and debug.